

rest-sample-cache-writer Kullanıcı Rehberi

Kısa kullanıcı rehberi

Bu rehber ilk kullanım içindir.

Amaç kısa ve nettir: PostgreSQL'den veriyi okuyup Redis'e hazır JSON snapshot olarak yazmak.

İçindekiler

1. Bu Proje Ne İşe Yarar?
2. Akış Nasıl Çalışır?
3. Ne Zaman Kullanılır?
4. Hızlı Başlangıç
5. Projection Mantığı
6. Önemli Ayarlar
7. İki Replica Nasıl Davranır?
8. Sık Hatalar

Bu Proje Ne İşe Yarar?

`rest-sample-cache-writer`, DB verisini Redis read model'e çeviren örnektir.

REST endpoint açmaz. Dubbo kullanmaz.

DB okuma Java tarafındadır. Redis yazma `java-rust-cache` ile Rust tarafındadır.

Akış Nasıl Çalışır?

Akış diyagramı:

flowchart LR

```
A["Scheduler"] --> B["Projection Job"]
B --> C["PostgreSQL + Hikari"]
C --> D["Java JSON Writer"]
D --> E["java-rust-cache"]
E --> F["Rust Redis Writer"]
F --> G["Redis Snapshot"]
```

Writer snapshot üretir. Reader bu snapshot'ı okur.

Bu ayrım read-heavy sistemlerde DB'yi hot path dışına çıkarır.

Ne Zaman Kullanılır?

Senaryo	Bu proje uygun mu?	Neden
DB'den read model üretilecek	Evet	Projection bazlı snapshot yazar.
REST API açılacak	Hayır	Bu proje HTTP server değildir.
Çok replica çalışacak	Evet	Projection bazlı lock vardır.
Her request'te DB query isteniyor	Hayır	Bu proje cache materializer'dır.
Redis Cluster veya Sentinel kullanılacak	Evet	<code>java-rust-cache</code> destekler.

Hızlı Başlangıç

PostgreSQL ve Redis'i hazırlayın.

Sonra writer'ı çalıştırın:

```
mvn -q package
java -jar target/rest-sample-cache-writer-0.1.0.jar
```

Tek seferlik snapshot için:

```
java "-Dsample.writer.run-once=true" -jar target/rest-sample-cache-writer-0.1.0.jar
```

Projection Mantığı

Projection, Redis'e yazılan ayrı bir read model parçasıdır.

Projection	Ne üretir?	Örnek kullanım
detail	Customer detail ve customerNo index	GET /customers/{id}
segment	Segment bazlı liste	Segment ekranı
status	Status bazlı liste	Aktif/pasif müşteri ekranı
campaign	Kampanya aday listesi	Kampanya seçimi
meta	Snapshot bilgisi	Cache hazır mı kontrolü

Her projection kendi interval, TTL ve lock değerine sahip olabilir.

Önemli Ayarlar

Property	Ne işe yarar?	Ne zaman değiştirilir?
sample.writer.projections=detail,segment,status,campaign,meta	Çalışacak projection listesidir.	Sadece ihtiyacınız olan read model'leri bırakın.
sample.writer.interval-ms=60000	Varsayılan yenileme aralığıdır.	Tüm projection'lar aynı sıklıkta yenilenecekse kullanın.
sample.writer.detail.interval-ms	Sadece detail projection aralığıdır.	Projection bazlı zamanlama istiyorsanız kullanın.
sample.writer.detail.cache-ttl-ms	detail verisinin Redis TTL değeridir.	Her zaman interval değerinden uzun olmalıdır.
sample.writer.scheduler-threads=2	Aynı anda kaç projection job çalışabilir.	Replica ve DB kapasitesine göre küçük tutun.
sample.writer.lock-ttl-ms=300000	Redis lock yaşam süresidir.	Job süresinden uzun olmalıdır.
sample.db.maximum-pool-size=2	Hikari DB connection üst limitidir.	DB kapasitesi ölçülmeden artırılmamalıdır.

İki Replica Nasıl Davranır?

```
Akış diyagramı:
sequenceDiagram
    participant R1 as Writer Replica 1
    participant R2 as Writer Replica 2
    participant Redis as Redis Lock
    participant DB as PostgreSQL

    R1->>Redis: detail lock alır
    R2->>Redis: campaign lock alır
    R1->>DB: detail verisini okur
    R2->>DB: campaign verisini okur
    R1->>Redis: detail snapshot yazar
    R2->>Redis: campaign snapshot yazar
```

İki replica aynı projection'ı aynı anda yazmaz.

Farklı projection'lar aynı anda farklı replica üzerinde çalışabilir.

Sık Hatalar

Belirti	Muhtemel neden	Çözüm
TTL uyarısı	TTL, interval değerinden kısa.	TTL değerini interval değerinden uzun yapın.
Lock alınamadı	Başka replica aynı projection'ı yazıyor.	Bu normaldir. Logları kontrol edin.

Belirti	Muhtemel neden	Çözüm
DB yavaşlıyor	Scheduler thread veya Hikari pool fazla.	<code>scheduler-threads</code> ve <code>maximum-pool-size</code> düşürün.
Reader <code>cache_not_ready</code> dönüyor	Writer henüz snapshot yazmadı.	Writer loglarında projection publish satırlarını kontrol edin.